

PYTHON SUMMER PLAN · DAY 21

Morning Launcher v2

JSON-Configured Routines

One config file. Many routines. Fire any of them with a name.

v1 worked — but it only knew one routine

This slide: why we are rebuilding the launcher, and what changes from yesterday's version.

BEFORE — v1

- ▶ **One routine, hardcoded.**

The actions lived inside the script. Want a second routine? Copy-paste and edit code.

- ▶ **No way to pick.**

Running the file always did the same thing.

- ▶ **Editing = touching code.**

Every tweak risked breaking the script.

AFTER — v2 (today)

- ▶ **Routines live in JSON.**

A plain text file holds many named routines — data, not code.

- ▶ **Pick by name.**

`python launcher.py study` runs the "study" routine.

- ▶ **Edit without coding.**

Add a routine by editing the JSON; the script never changes.

💡 TAKEAWAY

Same week's tools (`os_bridge` open/notify, schedule, clipboard) — now driven by a config file instead of buried in the code.

Routines live in a JSON file

This page: how a JSON file stores many routines as plain data, and how `json.load()` turns it into a Python dict.

```
# routines.json — a dict of named routines
{
  "study": {
    "label": "Study Mode",
    "actions": [
      {"type": "app", "value": "Spotify"},
      {"type": "url", "value": "https://duke.instructure.com"}
    ]
  }
}
```

```
import json, pathlib
cfg = json.loads(pathlib.Path("routines.json").read_text())
# cfg is now a normal Python dict
```

► **JSON ≈ Python dict/list.**

Curly braces are dicts, brackets are lists. It maps almost 1-to-1.

► **Each routine = a name.**

"study", "morning" — the key you type to run it.

► **actions is a list.**

An ordered list of small dicts. Each one says what to do and to what.

► **json.loads → dict.**

One call parses the text file into Python you can loop over.

One function dispatches every action

This page: the core loop. It walks a routine's action list and, based on each action's "type", calls the right os_bridge helper.

```
from os_bridge import launch_app, open_url, notify

def run_routine(cfg, name):
    routine = cfg[name]      # pick by name
    print(f'running {routine["label"]}')
    for act in routine["actions"]:
        kind = act["type"]
        if kind == "app": launch_app(act["value"])
        elif kind == "url": open_url(act["value"])
        elif kind == "notify": notify("Launcher", act["value"])
```

► **"type" picks the handler.**

The if/elif chain maps each action's type string to one helper call.

► **"value" is the argument.**

App name, URL, or message text — whatever that helper needs.

► **Reuses Day 16 os_bridge.**

launch_app / open_url / notify already exist; we just call them in order.

► **Add an action type later.**

New behaviour = one more elif. The JSON stays readable.

schedule — run a routine at a set time

This page: the schedule library lets a routine fire automatically at a clock time, without you launching it by hand.

```
import schedule, time

# every day at 07:30, run the 'morning' routine
schedule.every().day.at("07:30").do(run_routine, cfg, "morning")

while True:
    schedule.run_pending() # fire anything that is due
    time.sleep(1)         # check once a second
```

OUTPUT

```
[07:30:00] running Morning Startup
opening Spotify, mail, Canvas ...
```

▶ **.do()** takes args.

Extra arguments after the function get passed straight to `run_routine`.

▶ **Nothing fires on its own.**

`run_pending()` is what actually triggers due jobs — so it must be in a loop.

▶ **sleep(1) keeps it cheap.**

Without it the loop spins the CPU at 100%. One second is plenty.

▶ **Ctrl+C to stop.**

It is a blocking loop; kill it from the terminal when done.

pyperclip — read & write the clipboard

This page: pyperclip lets a routine put text on the clipboard (or read what is there) — handy for auto-pasting a snippet you use daily.

```
import pyperclip

pyperclip.copy("yr94@duke.edu") # put text on clipboard
text = pyperclip.paste()        # read what's there

# new action type, wired into run_routine:
elif kind == "clip": pyperclip.copy(act["value"])
```

OUTPUT

```
clipboard now holds: yr94@duke.edu
(paste anywhere with cmd+v)
```

► Two calls, that's it.

copy(text) writes, paste() reads. Both work with plain strings.

► Great for boilerplate.

Email, a sign-off, a common command — staged and ready to paste.

► Just another action.

Add "type": "clip" to a routine's JSON and the runner handles it.

► Needs the package.

pip3 install pyperclip — it shells out to the OS clipboard tool.

Choose the routine from the command line

This page: how a terminal argument selects which routine to run, with a fallback if you forget to name one.

```
import sys

def main():
    cfg = load_config("routines.json")
    name = sys.argv[1] if len(sys.argv) > 1 else "morning"
    if name not in cfg:
        print("unknown routine:", name); return
    run_routine(cfg, name)
```

```
$ python3 launcher.py study
```

► **sys.argv[1] = first word.**

Whatever you type after the filename becomes the routine name.

► **Default if blank.**

No argument → fall back to "morning" so a bare run still works.

► **Guard unknown names.**

Check the name is a key before using it; print a clear message otherwise.

► **Hotkey optional.**

Same main() can hang off a pynput global hotkey, like Day 17.

SUMMARY

Action types the runner understands

This page: the full vocabulary your JSON can use. Each row is one "type" value and the helper it triggers.

type	value means	handler	from
app	macOS app name	launch_app()	os_bridge (Day 16)
url	web address	open_url()	os_bridge (Day 16)
notify	banner text	notify()	os_bridge (Day 16)
clip	text to copy	pyperclip.copy()	Day 21 (new)

 TAKEAWAY

Adding a capability = one elif in run_routine + a new "type" string. The JSON files never need to know how it works.

TODAY'S EXERCISE · 75 MIN

Build launcher.py v2

Move your v1 routine into a routines.json file, then make the script run any named routine from it. Add at least one extra routine and one schedule trigger.

CONFIG

routines.json

Two+ routines (e.g. morning, study). Each has a label and an actions list of app / url / notify / clip dicts.

RUNNER

run_routine(cfg, name)

Loads JSON, looks up the routine, loops its actions, dispatches each by "type" to the right os_bridge / pyperclip helper.

TRIGGER

CLI + schedule

sys.argv picks the routine by name; add one schedule.every().day.at(...) job so 'morning' fires on its own.

Things that bite here

JSON is strict

Double quotes only, no trailing commas, no comments. A stray comma makes `json.load` raise `JSONDecodeError`. Validate the file if it won't parse.

Time format

`schedule.at()` wants 24-hour "HH:MM". "7:30" or "7:30 AM" will error — use "07:30".

Keep data and code apart

The script should know how to run actions, not which actions to run. That split is why adding a routine never touches Python.

Fail loud on bad names

Print the available routine names when the user types one that doesn't exist — saves guessing later.

End of Week 3 — and a bridge into Week 4

WEEK 4 · Day 24

File I/O

Today you read JSON with a one-liner. Day 24 slows down on pathlib + context managers — the proper way to read & write files.

WEEK 4 · Day 25

JSON + CSV

You'll formalise json.dump/load and add CSV. Same config idea, deeper. The launcher's routines.json is your first real config file.

CRUX

Config-driven everything

CRUX will load personality, routines, and panel layout from JSON exactly like this. Data-over-code is the pattern the whole project leans on.

Three weeks in: OOP → games → your laptop on autopilot. Next, it gets a window.